# Guía de Entregable — Semana 1

## Sistema de Administración Remota de Equipos (SARE)

**Equipo:** 4 personas · **Stack:** Python · **Metodología:** Kanban

---

## 0. Lo Primero: Crear el Repositorio

Antes de escribir una sola línea de código o documentación, el equipo necesita un lugar centralizado.

**Acción inmediata (Día 1, 30 minutos):**

1. El líder crea el repositorio desde su cuenta personal de GitHub: `sare-proyecto` (privado).
2. Agregar a los otros 3 miembros como colaboradores: *Repo → Settings → Collaborators → Add people* (Guía oficial).
3. Inicializar con esta estructura:

```
sare-proyecto/
├── docs/                    # Toda la documentación
│   ├── requisitos.md
│   ├── arquitectura.md
│   └── diccionario-datos.md
├── server/                  # Código de la consola (servidor)
│   ├── __init__.py
│   └── main.py
├── agent/                   # Código del agente (cliente)
│   ├── __init__.py
│   └── main.py
├── shared/                  # Código compartido (protocolo, constantes)
│   ├── __init__.py
│   └── protocol.py
├── tests/                   # Pruebas unitarias
├── .gitignore
├── README.md
└── requirements.txt
```

4. Regla de oro: **nadie trabaja en `main` directo**. Cada tarea se hace en una rama (`feature/nombre-tarea`) y se une con Pull Request (¿Qué es un Pull Request?).

**`.gitignore` mínimo para Python:**

```
__pycache__/
```

```
__pycache__/
*.pyc
.env
venv/
*.log
```

> **Referencia:** Git para principiantes (tutorial) · GitHub Docs en español

---

# 1. Documentación de Requisitos

Este documento es el corazón del entregable. El docente quiere ver que pensaron antes de codificar.

> **Referencia:** Fases del SDLC · IEEE 830 — Cómo escribir requisitos de software

## 1.1 Requisitos Funcionales (RF)

Son las cosas que el sistema **hace**. Redactarlos así:

| ID | Requisito | Prioridad | Protocolo |
|---|---|---|---|
| RF-01 | El servidor puede descubrir agentes conectados en la red local | Alta | TCP |
| RF-02 | El servidor puede enviar comando de bloqueo de pantalla a un agente específico | Alta | TCP |
| RF-03 | El servidor puede enviar comando de desbloqueo de pantalla | Alta | TCP |
| RF-04 | El servidor puede recibir stream de pantalla en vivo de un agente | Media | UDP |
| RF-05 | El servidor puede enviar comando de apagado/reinicio remoto | Alta | TCP |
| RF-06 | El servidor puede enviar mensaje emergente al agente | Baja | TCP |
| RF-07 | El agente se registra automáticamente al servidor al iniciar | Alta | TCP |
| RF-08 | El agente ejecuta comandos recibidos en el equipo local | Alta | Local |
| RF-09 | El agente puede transmitir captura de pantalla bajo demanda | Media | UDP |
| RF- | La consola muestra un dashboard con el estado de todos | | |

| | | | |
|---|---|---|---|
| RF-10 | La consola muestra un dashboard con el estado de todos los agentes | Media | — |

### 1.2 Requisitos No Funcionales (RNF)

Son las restricciones de calidad y diseño:

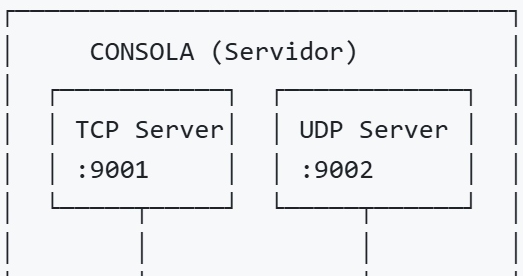
| ID | Requisito |
|---|---|
| RNF-01 | Los datos de pantalla se procesan en RAM; no se persisten en disco (Privacy by Design) |
| RNF-02 | La comunicación de comandos usa TCP para garantizar entrega |
| RNF-03 | El streaming de pantalla usa UDP para priorizar fluidez |
| RNF-04 | El agente debe consumir menos de 50MB de RAM en reposo |
| RNF-05 | El sistema debe funcionar en redes LAN (no requiere internet) |
| RNF-06 | El código debe seguir PEP8 y estar documentado con docstrings |
| RNF-07 | Todas las pruebas se ejecutan en entornos virtualizados (VMs), nunca en equipos reales |
| RNF-08 | El servidor soporta al menos 30 agentes concurrentes |

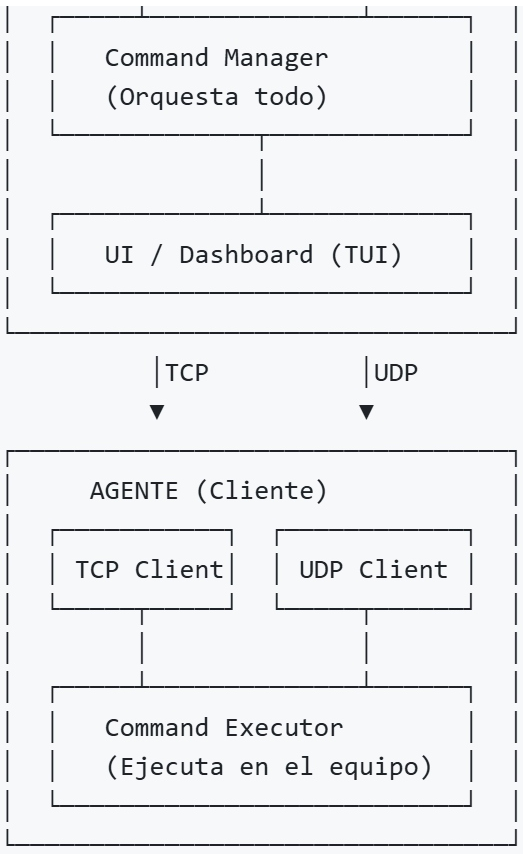
---

## 2. Arquitectura

> **Referencia:** Arquitectura Cliente-Servidor · Sockets en Python (docs oficiales) · TCP vs UDP explicado

### 2.1 Patrón: Cliente-Servidor con Protocolo Dual






## 2.2 Patrones de Diseño a Usar

> **Referencia:** Patrones de diseño en Python (Refactoring Guru)

**1. Command Pattern** — Para los comandos remotos. Cada comando (bloquear, apagar, mensaje) es una clase independiente que implementa una interfaz común. Esto permite agregar nuevos comandos sin tocar el código existente.

**2. Observer Pattern** — La consola "observa" el estado de los agentes. Cuando un agente se conecta/desconecta, la UI se actualiza automáticamente.

**3. Strategy Pattern** — Para elegir dinámicamente TCP o UDP según el tipo de dato que se va a transmitir.

## 2.3 Protocolo de Mensajes (Formato)

Definir un formato simple JSON para los mensajes entre servidor y agente:

```
{
  "type": "command",
  "action": "lock_screen",
  "payload": {},
  "timestamp": "2026-03-31T10:00:00Z",
  "request_id": "uuid-aqui"
}
```

Respuesta del agente:

```
{
  "type": "response",
  "request_id": "uuid-aqui",
  "status": "ok",
  "message": "Pantalla bloqueada"
}
```

---

# 3. Distribución del Equipo (Kanban + Roles)

## Tablero Kanban

Crear tablero gratuito en Trello (¿Qué es Kanban?).

Columnas: **Backlog → En Progreso (máx 2) → En Revisión → Terminado**

## Roles Semana 1

| Persona | Rol | Entregable Semana 1 |
|---|---|---|
| Persona 1 | Líder / Arquitecto | README.md, arquitectura.md, repo, coordinar PRs |
| Persona 2 | Documentación + Tests | requisitos.md, diccionario-datos.md, tests de protocolo |
| Persona 3 | Servidor (TCP) | Prototipo servidor TCP que acepta conexiones |
| Persona 4 | Agente (TCP) + Protocolo | shared/protocol.py + prototipo agente que se conecta |

---

# 4. Prototipo Funcional Mínimo (Semana 1)

El objetivo es demostrar que **dos programas Python se comunican por la red**. Nada más. No hace falta UI bonita, no hace falta stream de pantalla, no hace falta nada avanzado.

> **Referencia:** Socket Programming en Python (Real Python) · Módulo struct (docs oficiales)

## 4.1 Protocolo Compartido — `shared/protocol.py`

```
"""
Protocolo de comunicación SARE.
Define la serialización/deserialización de mensajes.
"""
import json
import struct
from datetime import datetime, timezone
from uuid import uuid4


def create_message(action: str, payload: dict = None) -> dict:
    """Crea un mensaje con formato estándar."""
    return {
        "type": "command",
        "action": action,
        "payload": payload or {},
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "request_id": str(uuid4()),
    }


def create_response(request_id: str, status: str, message: str = "") -> dict:
    """Crea una respuesta estándar del agente."""
    return {
        "type": "response",
        "request_id": request_id,
        "status": status,
        "message": message,
    }


def encode(msg: dict) -> bytes:
    """Serializa un mensaje a bytes con prefijo de longitud (4 bytes)."""
    data = json.dumps(msg).encode("utf-8")
    return struct.pack("!I", len(data)) + data


def decode(data: bytes) -> dict:
    """Deserializa bytes a un diccionario."""
    return json.loads(data.decode("utf-8"))


def recv_message(sock) -> dict:
    """Lee un mensaje completo desde un socket TCP."""
    # Leer los 4 bytes de longitud
    raw_len = _recv_exact(sock, 4)
    if not raw_len:
        return None
    msg_len = struct.unpack("!I", raw_len)[0]
    # Leer el cuerpo del mensaje
    raw_msg = _recv_exact(sock, msg_len)
    if not raw_msg:
        return None
```

```
    return decode(raw_msg)


def _recv_exact(sock, n: int) -> bytes:
    """Lee exactamente n bytes del socket."""
    data = b""
    while len(data) < n:
        chunk = sock.recv(n - len(data))
        if not chunk:
            return None
        data += chunk
    return data
```

## 4.2 Servidor Mínimo — `server/main.py`

```
"""
Servidor SARE — Prototipo Semana 1
Acepta conexiones de agentes y envía comandos básicos.
"""
import socket
import threading
from shared.protocol import (
    create_message, encode, recv_message
)

HOST = "0.0.0.0"
PORT = 9001
agents = {}  # {addr: socket}
agents_lock = threading.Lock()


def handle_agent(conn, addr):
    """Maneja la conexión con un agente individual."""
    print(f"[+] Agente conectado: {addr}")
    with agents_lock:
        agents[addr] = conn
    try:
        while True:
            msg = recv_message(conn)
            if msg is None:
                break
            print(f"    <- {addr}: {msg}")
    except (ConnectionResetError, BrokenPipeError):
        pass
    finally:
        print(f"[-] Agente desconectado: {addr}")
        with agents_lock:
            agents.pop(addr, None)
        conn.close()
```

```
def send_command(addr, action: str):
    """Envía un comando a un agente específico."""
    with agents_lock:
        conn = agents.get(addr)
    if conn:
        msg = create_message(action)
        conn.sendall(encode(msg))
        print(f"    -> {addr}: {action}")
    else:
        print(f"[!] Agente {addr} no encontrado")


def console_loop():
    """Loop interactivo para el administrador."""
    print("\nComandos disponibles:")
    print("  list           — Ver agentes conectados")
    print("  send <n> <cmd> — Enviar comando (ej: send 0 lock_screen)")
    print("  quit           — Salir\n")

    while True:
        try:
            line = input("sare> ").strip()
        except (EOFError, KeyboardInterrupt):
            break

        if line == "list":
            with agents_lock:
                for i, addr in enumerate(agents):
                    print(f"  [{i}] {addr}")
            if not agents:
                print("  (sin agentes conectados)")

        elif line.startswith("send "):
            parts = line.split(maxsplit=2)
            if len(parts) == 3:
                idx, action = int(parts[1]), parts[2]
                with agents_lock:
                    addrs = list(agents.keys())
                if 0 <= idx < len(addrs):
                    send_command(addrs[idx], action)

        elif line == "quit":
            break


def main():
    srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT))
    srv.listen(30)
    print(f"[*] Servidor SARE escuchando en {HOST}:{PORT}")

    # Aceptar conexiones en hilo aparte
    def accept_loop():
```

```
    def accept_loop():
        while True:
            conn, addr = srv.accept()
            t = threading.Thread(target=handle_agent, args=(conn, addr), daemon=True)
            t.start()

    threading.Thread(target=accept_loop, daemon=True).start()
    console_loop()
    srv.close()


if __name__ == "__main__":
    main()
```

## 4.3 Agente Mínimo — `agent/main.py`

```
"""
Agente SARE — Prototipo Semana 1
Se conecta al servidor y ejecuta comandos recibidos.
"""
import socket
import sys
from shared.protocol import (
    recv_message, encode, create_response
)

SERVER_HOST = "127.0.0.1"  # Cambiar a la IP del servidor en pruebas con VMs
SERVER_PORT = 9001

# Registro de ejecutores de comandos (Command Pattern simplificado)
COMMAND_HANDLERS = {}


def register_command(action: str):
    """Decorador para registrar un handler de comando."""
    def decorator(func):
        COMMAND_HANDLERS[action] = func
        return func
    return decorator


@register_command("lock_screen")
def handle_lock(payload: dict) -> str:
    # Aquí iría la lógica real de bloqueo (ctypes en Windows, etc.)
    print("  [!] PANTALLA BLOQUEADA (simulado)")
    return "Pantalla bloqueada"


@register_command("unlock_screen")
def handle_unlock(payload: dict) -> str:
    print("  [✓] Pantalla desbloqueada (simulado)")
    return "Pantalla desbloqueada"
```

```
    return "Pantalla desbloqueada"


@register_command("ping")
def handle_ping(payload: dict) -> str:
    return "pong"


def main():
    host = sys.argv[1] if len(sys.argv) > 1 else SERVER_HOST
    port = int(sys.argv[2]) if len(sys.argv) > 2 else SERVER_PORT

    print(f"[*] Conectando a {host}:{port}...")
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((host, port))
    print(f"[+] Conectado al servidor")

    try:
        while True:
            msg = recv_message(sock)
            if msg is None:
                print("[-] Servidor cerró la conexión")
                break

            print(f"  <- Comando recibido: {msg['action']}")
            action = msg.get("action", "")
            handler = COMMAND_HANDLERS.get(action)

            if handler:
                result = handler(msg.get("payload", {}))
                status = "ok"
            else:
                result = f"Comando desconocido: {action}"
                status = "error"

            resp = create_response(msg["request_id"], status, result)
            sock.sendall(encode(resp))
    except (ConnectionResetError, BrokenPipeError):
        print("[-] Conexión perdida")
    finally:
        sock.close()


if __name__ == "__main__":
    main()
```

---

# 5. Cómo Probar el Prototipo

**Opción rápida (misma máquina):**

1. Abrir terminal 1 

1. Abrir terminal 1: `python -m server.main`
2. Abrir terminal 2: `python -m agent.main`
3. En terminal 1, escribir `list` para ver al agente, luego `send 0 lock_screen`

**Opción correcta (VMs — para la siguiente semana):**

- Crear 2 VMs con VirtualBox (Windows o Linux)
- Red en modo "Red Interna" o "Host-Only" (Guía de redes en VirtualBox)
- Servidor en VM1, Agente en VM2
- Cambiar `SERVER_HOST` a la IP de VM1

> **Referencia:** Qué es un Homelab · Privacy by Design (7 principios)

---

## 6. README.md para el Repositorio

```
# SARE — Sistema de Administración Remota de Equipos

## Descripción
Sistema cliente-servidor para administración remota de equipos
en redes LAN educativas. Permite al administrador enviar comandos
(bloqueo de pantalla, reinicio, mensajes) y recibir stream de
pantalla en tiempo real.

## Arquitectura
- **Servidor (Consola):** Orquesta comandos vía TCP, recibe stream vía UDP
- **Agente (Cliente):** Ejecuta comandos localmente, transmite pantalla

## Stack Tecnológico
- Python 3.10+
- Sockets TCP/UDP (librería estándar)
- Threading para concurrencia

## Estructura del Proyecto
(copiar el árbol de la sección 0)

## Cómo ejecutar
### Servidor
python -m server.main

### Agente
python -m agent.main [IP_SERVIDOR] [PUERTO]

## Equipo
| Nombre | Rol |
|--------|-----|
| ...    | Líder / Arquitecto |
| ...    | Documentación + Tests |
| ...    | Servidor (TCP) |
```

```
| ...    | Agente + Protocolo |

## Licencia
Proyecto educativo — SENA 2026
```

---

## 7. Checklist de Entrega — Semana 1

- [ ] Repositorio creado en GitHub con estructura de carpetas
- [ ] Los 4 miembros agregados como colaboradores con al menos 1 commit cada uno
- [ ] README.md completo con descripción, arquitectura y equipo
- [ ] `docs/requisitos.md` con tablas RF y RNF
- [ ] `docs/arquitectura.md` con diagrama y patrones de diseño
- [ ] `shared/protocol.py` funcional con encode/decode
- [ ] `server/main.py` — acepta conexiones y envía comandos
- [ ] `agent/main.py` — se conecta y ejecuta comandos simulados
- [ ] Demostración funcional: servidor envía `lock_screen`, agente responde
- [ ] Tablero Kanban creado en Trello con tareas asignadas

---

## 8. Links de Referencia Rápida

| Tema | Link |
|---|---|
| Git básico | git-scm.com/book/es |
| GitHub Docs (español) | docs.github.com/es |
| SDLC | aws.amazon.com/es/what-is/sdlc |
| Kanban | atlassian.com/es/agile/kanban |
| Sockets en Python | docs.python.org/es/3/howto/sockets |
| Socket Programming (Real Python) | realpython.com/python-sockets |
| TCP vs UDP | cloudflare.com/es-es/learning/ddos/glossary/tcp-ip |
| Patrones de diseño (Python) | refactoring.guru/es/design-patterns/python |
| Privacy by Design | wikipedia.org/wiki/Privacidad_por_diseño |
| VirtualBox (Homelabs) | virtualbox.org |
| PEP 8 (estilo Python) | peps.python.org/pep-0008 |

| | |
|---|---|
| Prompt Engineering | platform.openai.com/docs/guides/prompt-engineering |